



Date: __ / __ / ____

Experiment 2

2. a. Write a C program that implements the Merge Sort algorithm to sort an array of integers in ascending order. Create a function **mergeSort** that takes an array and its size as parameters and sorts the array using the Merge Sort technique. Additionally, create a function **merge** to merge two sorted halves of the array during the sorting process. Finally, print the sorted array.

Program

```
#include <stdio.h>
#include <stdlib.h>

// Function to merge two sorted halves of an array
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int L[n1], R[n2];

    // Copy data to temporary arrays L[] and R[]
    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    // Merge the temporary arrays back into arr[left..right]
    i = 0;
    j = 0;
    k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
}
```



```
// Copy the remaining elements of L[], if there are any
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

// Copy the remaining elements of R[], if there are any
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}

// Function to perform Merge Sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        // Same as (left+right)/2, but avoids overflow for large left and right
        int mid = left + (right - left) / 2;

        // Recursively sort the first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int n;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}
```



```
// Perform Merge Sort
mergeSort(arr, 0, n - 1);

// Print the sorted array
printf("Sorted array: ");
printArray(arr, n);

return 0;
}
```

Sample Output:

Enter the number of elements: 5

Enter the elements:

5

4

3

2

1

Sorted array: 1 2 3 4 5